WEEK 7

Navigation Between Screens

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

Real apps have many screens: a list, a detail page, settings, a profile. This week you'll learn **Navigation Compose**: how to move between screens, pass data, control the back button, and build a **bottom navigation bar**. You'll finish with a **multi-screen Quiz app**.

Day	Topic	You will be able to...
Day 1	Navigation basics	Set up NavHost and move between two screens
Day 2	Passing data between screens	Send IDs and values with type-safe routes
Day 3	The back stack	Control Back, popUpTo and single-top navigation
Day 4	Bottom navigation	Build tabs with a NavigationBar that keeps each tab's state
Day 5	Structuring navigation in an app	Organise routes, graphs and top bars like a team
Day 6	Weekly project	Build a multi-screen Quiz app

Tip: Compose code is **compile-checked**. The Navigation library itself couldn't be downloaded here, so navigation calls were checked against stand-ins that copy its API (Navigation 2.8+, type-safe routes). Pure Kotlin logic was run, with real output.

WEEK 7 • DAY 1

Navigation Basics

Today's goal: Add Navigation Compose to a project and navigate between two screens.

1. The parts of navigation

Part	What it is	Real-life comparison
Route	The address of a screen (a @Serializable object or class)	A house address
NavController	Moves between screens and remembers the history	The driver
NavHost	Shows the current screen and lists all the routes	The map of the city
Back stack	The list of screens visited; Back removes the top one	A pile of plates

2. Add the dependencies

gradle/libs.versions.toml (add these lines)

```
[versions]
navigationCompose = "2.8.5"        # use the version your team/project uses
kotlinSerialization = "2.0.21"     # must match your Kotlin version

[libraries]
androidx-navigation-compose = { group = "androidx.navigation", name = "navigation-compose", version.ref = "navigationCompose" }
kotlinx-serialization-json = { group = "org.jetbrains.kotlinx", name = "kotlinx-serialization-json", version = "1.7.3" }

[plugins]
kotlin-serialization = { id = "org.jetbrains.kotlin.plugin.serialization", version.ref = "kotlinSerialization" }
```

app/build.gradle.kts (Android only)

```
plugins {
    // ... existing plugins
    alias(libs.plugins.kotlin.serialization)
}

dependencies {
    // ... existing dependencies
    implementation(libs.androidx.navigation.compose)
    implementation(libs.kotlinx.serialization.json)
}
```

Careful: Version numbers must work with your Android Studio, Gradle plugin and Kotlin version (remember the AAR metadata error). In a team project, navigation is almost always set up already. Just use it.

3. Define routes

Routes.kt

```
@Serializable
object HomeRoute

@Serializable
object AboutRoute
```

A route with no data is an **object**. **@Serializable** lets Navigation turn the route into a path internally. You never write strings like "home" by hand, so you can't make typos.

4. Build the NavHost

Two screens connected

```
@Serializable object HomeRoute
@Serializable object AboutRoute

@Composable
fun AppNavHost() {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = HomeRoute) {
        composable<HomeRoute> {
            HomeScreen(onAboutClick = { navController.navigate(AboutRoute) })
        }
        composable<AboutRoute> {
            AboutScreen(onBack = { navController.popBackStack() })
        }
    }
}

@Composable
fun HomeScreen(onAboutClick: () -> Unit) {
    Column(
        modifier = Modifier.fillMaxSize().padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Home", style = MaterialTheme.typography.headlineMedium)
        Spacer(Modifier.height(16.dp))
        Button(onClick = onAboutClick) { Text("About this app") }
    }
}

@Composable
fun AboutScreen(onBack: () -> Unit) {
    Column(modifier = Modifier.fillMaxSize().padding(24.dp)) {
        Text("About", style = MaterialTheme.typography.headlineMedium)
        Text("Made by Anjana while learning Android.")
        Spacer(Modifier.height(16.dp))
        OutlinedButton(onClick = onBack) { Text("Back") }
    }
}
```

Code	Meaning
rememberNavController()	Creates the NavController and keeps it across recompositions
NavHost(navController, startDestination = HomeRoute)	The first screen shown is HomeRoute
composable<HomeRoute> { ... }	"When the route is HomeRoute, show this composable"
navController.navigate(AboutRoute)	Push AboutRoute on top of the back stack
navController.popBackStack()	Remove the top screen (the same as pressing Back)

5. The golden rule: screens don't know about navigation

Look at HomeScreen: it receives **onAboutClick: () -> Unit**, not the NavController. The screen only says *"the About button was clicked"*. The NavHost decides *where* to go.

Passing lambdas (good)	Passing NavController (avoid)
The screen can be previewed with { }	Previews need a fake NavController

Passing lambdas (good)	Passing NavController (avoid)
The screen can be reused in another flow	The screen is tied to one navigation graph
All navigation is visible in one file	Navigation is hidden inside many screens

6. Connect it to MainActivity

MainActivity.kt (Android only)

```
setContent {
    Week7PracticeTheme {
        Surface(modifier = Modifier.fillMaxSize()) {
            AppNavHost()
        }
    }
}
```

Now press the system **Back** button on the About screen: Navigation handles it automatically and returns to Home.

Practice Exercises

1. Add a third screen, **ContactRoute**, reachable from a button on HomeScreen.

2. Give AboutScreen a TopAppBar with a back arrow (navigationIcon) instead of the Back button.

3. Add a preview for HomeScreen and AboutScreen. Notice that you don't need a NavController for them.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What does NavHost do?

Q2. What does navController.navigate(AboutRoute) do?

Q3. Why is HomeRoute an object?

Q4. Why do screens receive lambdas instead of the NavController?

Q5. Which plugin do type-safe routes need?

WEEK 7 • DAY 2

Passing Data Between Screens

Today's goal: Send values to a screen with route arguments, and follow the "pass IDs, not objects" rule.

1. Routes with arguments

A route that carries data is a **data class**. Its properties are the arguments.

Route with arguments

```
@Serializable
object ProductListRoute

@Serializable
data class ProductDetailRoute(val productId: Int)

@Serializable
data class SearchRoute(val query: String, val category: String? = null, val page: Int = 1)
```

Argument	Meaning
val productId: Int	Required. You must pass it
val category: String? = null	Optional and nullable
val page: Int = 1	Optional, with a default value

2. Sending and reading arguments

List to detail

```
data class Product(val id: Int, val name: String, val price: Int)

val allProducts = listOf(
    Product(1, "Notebook", 45),
    Product(2, "Pen set", 120),
    Product(3, "Backpack", 899)
)

@Serializable object ProductListRoute
@Serializable data class ProductDetailRoute(val productId: Int)

@Composable
fun ShopNavHost() {
    val navController = rememberNavController()
    NavHost(navController = navController, startDestination = ProductListRoute) {
        composable<ProductListRoute> {
            ProductListScreen(
                products = allProducts,
                onProductClick = { id -> navController.navigate(ProductDetailRoute(productId = id)) }
            )
        }
        composable<ProductDetailRoute> { backStackEntry ->
            val route: ProductDetailRoute = backStackEntry.toRoute()
            val product = allProducts.find { it.id == route.productId }
            ProductDetailScreen(product = product, onBack = { navController.popBackStack() })
        }
    }
}
```

```
@Composable
fun ProductListScreen(products: List<Product>, onProductClick: (Int) -> Unit) {
    LazyColumn {
        items(products, key = { it.id }) { product ->
            Text(
                text = "${product.name} - Rs ${product.price}",
                modifier = Modifier
                    .fillMaxWidth()
                    .clickable { onProductClick(product.id) }
                    .padding(16.dp)
            )
        }
    }
}

@Composable
fun ProductDetailScreen(product: Product?, onBack: () -> Unit) {
    Column(modifier = Modifier.padding(24.dp)) {
        if (product == null) {
            Text("Product not found")
        } else {
            Text(product.name, style = MaterialTheme.typography.headlineMedium)
            Text("Rs ${product.price}", style = MaterialTheme.typography.titleLarge)
        }
        Spacer(Modifier.height(16.dp))
        Button(onClick = onBack) { Text("Back") }
    }
}
```

backStackEntry.toRoute() turns the current back stack entry back into your ProductDetailRoute object, so **route.productId** is a normal Int. Navigation checks the types for you.

3. Pass IDs, not whole objects

AVOID: putting a whole object in the route

```
@Serializable data class ProductDetailRoute(val product: Product)   // big, can go stale, needs Product to be @Serializable
```

Pass the ID	Why
ProductDetailRoute(productId = 3)	Small and simple. The detail screen loads the **latest** data using the ID (from a ViewModel or repository)
Not ProductDetailRoute(product)	Objects can be large, can become outdated, and are lost or broken when Android restores the app

This is the official recommendation from Google, and teams follow it. In Week 8, the detail screen's **ViewModel** will read the ID and load the product.

4. Optional arguments

Using default values

```
@Serializable
data class OrdersRoute(val filter: String = "all", val highlightOrderId: Int? = null)

fun navigationCalls(navController: NavHostController) {
    navController.navigate(OrdersRoute())                                    // filter = "all"
    navController.navigate(OrdersRoute(filter = "delivered"))
    navController.navigate(OrdersRoute(filter = "pending", highlightOrderId = 1024))
}
```

5. Sending a result back (optional)

Sometimes screen B must return a value to screen A, for example a selected colour. One simple way is the previous screen's **savedStateHandle**:

Returning a value (Android only)

```
// In screen B (the picker), before going back:
navController.previousBackStackEntry?.savedStateHandle?.set("selected_color", "Green")
navController.popBackStack()

// In screen A's composable<...> block:
val selected = backStackEntry.savedStateHandle
    .getStateFlow("selected_color", "None")
    .collectAsStateWithLifecycle()
```

Tip: In team apps, results are often shared through a ViewModel or repository instead. Use whichever pattern your team already uses.

Practice Exercises

1. Create **StudentDetailRoute(val rollNumber: Int)** and a list screen of 5 students that opens the detail screen for the tapped student.

2. Create **ArticleRoute(val articleId: Int, val fromNotification: Boolean = false)**. Show a banner on the article screen when fromNotification is true.

3. In the product example, show "Product not found" for an ID that doesn't exist, by navigating to ProductDetailRoute(99).

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. How do you make a route that carries data?

Q2. How do you read the arguments in the destination?

Q3. Why pass an ID instead of a whole object?

Q4. How do you make an argument optional?

Q5. What happens if you pass a String where the route expects an Int?

WEEK 7 • DAY 3

The Back Stack

Today's goal: Control the back stack with popBackStack, popUpTo, inclusive and launchSingleTop, and handle Back yourself.

1. Visualising the back stack

Sketch: What happens to the stack

```
Start:          navigate(B):     navigate(C):     Back:            Back:
+---+           +---+            +---+            +---+            +---+
| A |           | B |            | C |            | B |            | A |
+---+           +---+            +---+            +---+            +---+
                | A |            | B |            | A |
                +---+            +---+            +---+
                                 | A |
                                 +---+
Pressing Back on A (the last screen) closes the app.
```

2. Going back

Call	What it does
navController.popBackStack()	Removes the top screen. Returns false if there's nothing to go back to
navController.navigateUp()	Goes "up" in the app's hierarchy. Use it for the top app bar's back arrow
System Back button / gesture	Handled automatically by NavHost: pops the top screen

3. popUpTo: removing screens from the stack

After logging in, the user shouldn't be able to press Back and return to the login screen. Use **popUpTo** to remove screens while navigating:

Login flow with popUpTo

```
@Serializable object SplashRoute
@Serializable object LoginRoute
@Serializable object HomeRoute

@Composable
fun AuthNavHost() {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = SplashRoute) {
        composable<SplashRoute> {
            SplashScreen(onFinished = {
                navController.navigate(LoginRoute) {
                    popUpTo<SplashRoute> { inclusive = true }      // remove the splash screen too
                }
            })
        }
        composable<LoginRoute> {
            LoginScreen(onLoginSuccess = {
                navController.navigate(HomeRoute) {
                    popUpTo<LoginRoute> { inclusive = true }       // Back from Home closes the app
                }
            })
```

```
        }
        composable<HomeRoute> {
            Text("Welcome home!", modifier = Modifier.padding(24.dp))
        }
    }
}

@Composable
fun SplashScreen(onFinished: () -> Unit) {
    LaunchedEffect(Unit) {
        delay(1500)               // show the splash for 1.5 seconds
        onFinished()
    }
    Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
        Text("My App", style = MaterialTheme.typography.displaySmall)
    }
}

@Composable
fun LoginScreen(onLoginSuccess: () -> Unit) {
    Column(modifier = Modifier.padding(24.dp)) {
        Text("Login", style = MaterialTheme.typography.headlineMedium)
        Button(onClick = onLoginSuccess) { Text("Log in") }
    }
}
```

Sketch: Effect of popUpTo inclusive

```
Without popUpTo:  [Splash, Login, Home]  -> Back from Home goes to Login (bad!)
With popUpTo:     [Home]                 -> Back from Home closes the app (good)
```

Option	Meaning
popUpTo<LoginRoute> { }	Remove every screen **above** LoginRoute before navigating
inclusive = true	Also remove LoginRoute itself
launchSingleTop = true	Don't add a second copy if the destination is already on top (prevents double taps opening the screen twice)

LaunchedEffect(Unit) { } runs a block of code once when the composable first appears. Here it waits with **delay** (a coroutine function) and then navigates. You'll learn coroutines properly in Week 9.

4. Logging out: clearing everything

Clear the back stack on logout

```
@Serializable object LoginRoute
@Serializable object HomeRoute
@Serializable object SettingsRoute

fun logout(navController: NavHostController) {
    navController.navigate(LoginRoute) {
        popUpTo(navController.graph.findStartDestination().id) { inclusive = true }
        launchSingleTop = true
    }
}
```

5. Handling Back yourself

Ask before leaving a form (Android only)

```
@Composable
fun EditNoteScreen(hasUnsavedChanges: Boolean, onExit: () -> Unit) {
    var showDialog by remember { mutableStateOf(false) }

    BackHandler(enabled = hasUnsavedChanges) {     // only intercept Back when there are changes
        showDialog = true
    }

    if (showDialog) {
        AlertDialog(
            onDismissRequest = { showDialog = false },
            title = { Text("Discard changes?") },
            confirmButton = { TextButton(onClick = onExit) { Text("Discard") } },
            dismissButton = { TextButton(onClick = { showDialog = false }) { Text("Keep editing") } }
        )
    }
    // ... the form UI
}
```

Practice Exercises

1. Build an onboarding flow: Welcome → Choose language → Home. After Home opens, Back should close the app.

2. Add a Settings screen with a Log out button that uses the logout() function and returns to Login with an empty back stack.

3. Make a button that navigates to Settings with launchSingleTop = true, then tap it quickly twice. Press Back once. Where are you?

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What happens when you press Back on the last screen in the stack?

Q2. What does popUpTo { inclusive = true } do?

Q3. Why use launchSingleTop?

Q4. Which function should the top bar's back arrow call?

Q5. What is BackHandler for?

WEEK 7 • DAY 4

Bottom Navigation

Today's goal: Build a bottom navigation bar connected to Navigation, with the correct selected tab and saved tab state.

1. How bottom navigation works

Sketch: Tabs and the NavHost

```
+--------------------------------+
|                                |
|   NavHost shows the current    |   <- composable<FeedRoute>, <SearchRoute>, <ProfileRoute>
|   tab's screen                 |
|                                |
+--------------------------------+
| (Home)    (Search)   (Person)  |   <- NavigationBar: tapping a tab = navigate(route)
|  Feed      Search    Profile   |
+--------------------------------+
```

Each tab is a **top-level destination**. Tapping a tab navigates to its route. The highlighted tab is worked out from the **current back stack entry**, so it's always correct, even after pressing Back.

2. The complete code

MainScreen with bottom navigation

```
@Serializable object FeedRoute
@Serializable object SearchRoute
@Serializable object ProfileRoute

data class TopLevelDestination<T : Any>(
    val route: T,
    val label: String,
    val icon: ImageVector
)

val topLevelDestinations = listOf(
    TopLevelDestination(FeedRoute, "Feed", Icons.Default.Home),
    TopLevelDestination(SearchRoute, "Search", Icons.Default.Search),
    TopLevelDestination(ProfileRoute, "Profile", Icons.Default.Person)
)

@Composable
fun MainScreen() {
    val navController = rememberNavController()
    val navBackStackEntry by navController.currentBackStackEntryAsState()
    val currentDestination = navBackStackEntry?.destination

    Scaffold(
        bottomBar = {
            NavigationBar {
                topLevelDestinations.forEach { destination ->
                    val selected = currentDestination?.hierarchy?.any {
                        it.hasRoute(destination.route::class)
                    } == true
                    NavigationBarItem(
                        selected = selected,
                        onClick = {
```

```
                            navController.navigate(destination.route) {
                                popUpTo(navController.graph.findStartDestination().id) {
                                    saveState = true
                                }
                                launchSingleTop = true
                                restoreState = true
                            }
                        },
                        icon = { Icon(destination.icon, contentDescription = null) },
                        label = { Text(destination.label) }
                    )
                }
            }
        }
    ) { innerPadding ->
        NavHost(
            navController = navController,
            startDestination = FeedRoute,
            modifier = Modifier.padding(innerPadding)
        ) {
            composable<FeedRoute> { TabContent("Feed") }
            composable<SearchRoute> { TabContent("Search") }
            composable<ProfileRoute> { TabContent("Profile") }
        }
    }
}

@Composable
fun TabContent(title: String) {
    Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
        Text(title, style = MaterialTheme.typography.headlineMedium)
    }
}
```

3. Understand the important lines

Code	What it does
currentBackStackEntryAsState()	State that updates every time the current screen changes, so the bar recomposes
destination.hierarchy.any { it.hasRoute(...) }	Is the current screen this tab (or inside this tab's nested graph)?
popUpTo(startDestination) { saveState = true }	Don't build an endless stack of tabs. Go back to the start tab, saving the state of the tab you're leaving
launchSingleTop = true	Tapping the current tab again doesn't add another copy
restoreState = true	Coming back to a tab restores its saved state (for example, the scroll position)
NavHost(modifier = Modifier.padding(innerPadding))	Keeps the screens above the bottom bar

hierarchy, **hasRoute** and **findStartDestination** are special extension functions. If they're red, press Alt + Enter: Android Studio imports them from **NavDestination.Companion** and **NavGraph.Companion**.

> **Tip:** This snippet is copied almost word for word in real apps, including Google's "Now in Android" sample. You don't need to memorise it, but understand what each option does.

4. Hiding the bottom bar on some screens

Show the bar only on top-level screens

```
@Serializable object FeedRoute
@Serializable data class PostDetailRoute(val postId: Int)

@Composable
fun shouldShowBottomBar(navController: NavHostController): Boolean {
    val entry by navController.currentBackStackEntryAsState()
    val destination = entry?.destination ?: return true
    return !destination.hasRoute(PostDetailRoute::class)
}
```

Use it as `bottomBar = { if (shouldShowBottomBar(navController)) NavigationBar { ... } }`. Detail screens usually hide the bottom bar to give more space.

5. Bottom bar vs navigation rail vs drawer

Component	Use when
NavigationBar (bottom)	3 to 5 top-level destinations on a phone
NavigationRail (side)	Tablets and large screens
ModalNavigationDrawer	Many destinations, or less important ones (settings, help)

Practice Exercises

1. Add a fourth tab, **Notifications**, with Icons.Default.Notifications.

2. Replace TabContent in the Feed tab with a LazyColumn of 30 items. Scroll down, switch tabs and come back: is the scroll position restored?

3. Add a detail screen to the Feed tab and hide the bottom bar on it.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. How does the bar know which tab to highlight?

Q2. What does saveState = true do?

Q3. Why use popUpTo the start destination when switching tabs?

Q4. Why does the NavHost get Modifier.padding(innerPadding)?

Q5. How many destinations suit a bottom navigation bar?

WEEK 7 • DAY 5

Structuring Navigation in an App

Today's goal: Organise navigation the way teams do: routes in one place, nested graphs and top bars per screen.

1. A typical team structure

Sketch: Where navigation code lives

```
app/
 |-- navigation/
 |    |-- AppNavHost.kt        <- the NavHost: connects every route to its screen
 |    +-- TopLevelDestinations.kt
 |-- feature/
 |    |-- home/
 |    |    |-- HomeRoute.kt    <- @Serializable object HomeRoute (sometimes in HomeScreen.kt)
 |    |    +-- HomeScreen.kt   <- receives lambdas: onItemClick, onSettingsClick
 |    +-- settings/ ...
```

2. Feature navigation extension functions

As apps grow, each feature adds its own destinations with a small **extension function** on NavGraphBuilder. AppNavHost then stays short and readable.

Feature-based navigation

```
@Serializable object NotesRoute
@Serializable data class NoteDetailRoute(val noteId: Int)

// feature/notes/NotesNavigation.kt
fun NavGraphBuilder.notesScreens(onOpenNote: (Int) -> Unit, onBack: () -> Unit) {
    composable<NotesRoute> {
        NotesScreen(onNoteClick = onOpenNote)
    }
    composable<NoteDetailRoute> { entry ->
        val route: NoteDetailRoute = entry.toRoute()
        NoteDetailScreen(noteId = route.noteId, onBack = onBack)
    }
}

// navigation/AppNavHost.kt
@Composable
fun AppNavHost(navController: NavHostController = rememberNavController()) {
    NavHost(navController = navController, startDestination = NotesRoute) {
        notesScreens(
            onOpenNote = { id -> navController.navigate(NoteDetailRoute(id)) },
            onBack = { navController.popBackStack() }
        )
        // settingsScreens(...), profileScreens(...)
    }
}

@Composable
fun NotesScreen(onNoteClick: (Int) -> Unit) {
    LazyColumn {
        items((1..10).toList(), key = { it }) { id ->
            Text("Note $id", Modifier.fillMaxWidth().clickable { onNoteClick(id) }.padding(16.dp))
        }
    }
}

@Composable
fun NoteDetailScreen(noteId: Int, onBack: () -> Unit) {
    Column(Modifier.padding(16.dp)) {
        Text("Note $noteId", style = MaterialTheme.typography.headlineSmall)
        TextButton(onClick = onBack) { Text("Back") }
    }
}
```

3. Nested graphs

A **nested graph** groups related screens, for example all the screens of a sign-up flow. You can pop the whole group at once.

A nested sign-up graph

```
@Serializable object SignUpGraph              // the graph's own route
@Serializable object EnterNameRoute
@Serializable object EnterEmailRoute
@Serializable object HomeRoute

@Composable
fun SignUpNavHost() {
    val navController = rememberNavController()
    NavHost(navController = navController, startDestination = SignUpGraph) {
        navigation<SignUpGraph>(startDestination = EnterNameRoute) {
            composable<EnterNameRoute> {
                Button(onClick = { navController.navigate(EnterEmailRoute) }) { Text("Next") }
            }
            composable<EnterEmailRoute> {
                Button(onClick = {
                    navController.navigate(HomeRoute) {
                        popUpTo<SignUpGraph> { inclusive = true }   // remove the whole sign-up flow
                    }
                }) { Text("Finish") }
            }
        }
        composable<HomeRoute> { Text("Home") }
    }
}
```

4. Each screen owns its top bar

A screen with its own Scaffold and TopAppBar

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun SettingsScreen(onBack: () -> Unit) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Settings") },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Back")
                    }
                }
            )
        }
    ) { innerPadding ->
        Column(modifier = Modifier.padding(innerPadding).padding(16.dp)) {
            Text("Notifications")
            Text("Language")
        }
    }
}
```

Most teams give **each screen its own Scaffold and top bar** (with its own title and actions). The shared bottom bar, if any, lives in the outer Scaffold around the NavHost, as on Day 4.

5. Deep links (a quick look)

A **deep link** opens a specific screen from outside the app, for example from a notification or a web URL like *myshop.com/product/3*. Navigation supports this with **deepLinks = listOf(navDeepLink<ProductDetailRoute>(basePath = "..."))** in composable<...>, plus an intent filter in the manifest. You'll meet it in team projects. Read the official docs when you need it.

6. Navigation checklist for code review

Check	Why
Screens receive lambdas, not NavController	Previewable, reusable, testable
Routes pass IDs, not objects	Fresh data, no stale or huge arguments
Login, onboarding and splash use popUpTo inclusive	Back doesn't return to them
Repeated taps use launchSingleTop	No duplicate screens
Bottom bar uses saveState / restoreState	Tabs keep their state

Practice Exercises

1. Move the Day 2 product screens into a **NavGraphBuilder.shopScreens(...)** extension function and call it from AppNavHost.

2. Build a 3-step checkout as a nested graph (Address → Payment → Confirm) that is removed completely when the order is placed.

3. Open your team's navigation code and find: the start destination, one route with arguments and one popUpTo.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why use NavGraphBuilder extension functions per feature?

Q2. What is a nested graph?

Q3. How do you remove a whole nested graph from the back stack?

Q4. Where does a screen's TopAppBar usually live?

Q5. What is a deep link?

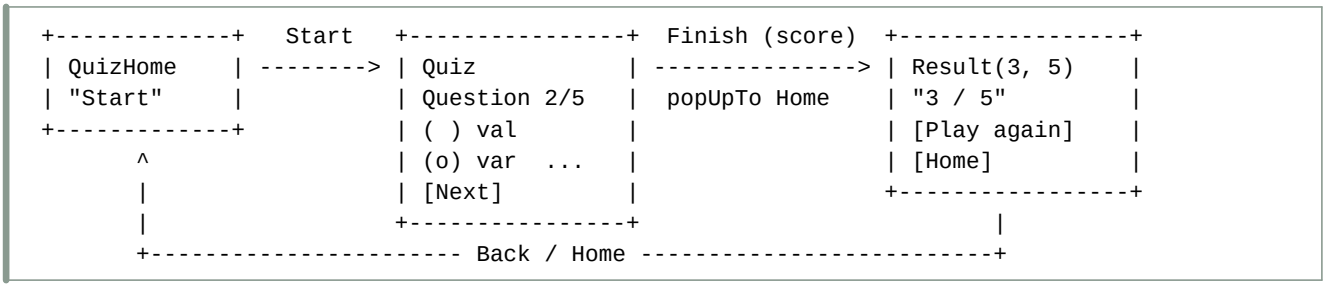
WEEK 7 • DAY 6

Weekly Project: Multi-Screen Quiz App

Today's goal: Build a three-screen quiz app with arguments, popUpTo and saved state.

1. The screens

Sketch: Navigation flow



2. The quiz logic, as plain Kotlin

QuizLogic.kt (tested with real output)

```
data class Question(
    val text: String,
    val options: List<String>,
    val correctIndex: Int
)

val kotlinQuiz = listOf(
    Question("Which keyword declares a read-only variable?", listOf("var", "val", "let", "const"), 1),
    Question("Which function runs when an Activity is created?", listOf("onStart", "onResume", "onCreate", "onShow"), 2),
    Question("Which layout places children vertically?", listOf("Row", "Box", "Column", "Grid"), 2),
    Question("What makes a value survive rotation?", listOf("remember", "rememberSaveable", "var", "lazy"), 1),
    Question("Which composable shows a long list efficiently?", listOf("Column", "LazyColumn", "Row", "Card"), 1)
)

fun score(answers: List<Int>, questions: List<Question>): Int =
    questions.indices.count { i -> answers.getOrNull(i) == questions[i].correctIndex }

fun resultMessage(score: Int, total: Int): String {
    val percent = score * 100 / total
    return when {
        percent == 100 -> "Perfect! You're a Kotlin star."
        percent >= 60 -> "Good job! Review the ones you missed."
        else -> "Keep practising. You'll get there!"
    }
}

fun main() {
    val perfect = listOf(1, 2, 2, 1, 1)
    val some = listOf(1, 0, 2, 0, 1)
    val none = listOf(0, 0, 0, 0, 0)
    for (answers in listOf(perfect, some, none)) {
        val s = score(answers, kotlinQuiz)
        println("$s/${kotlinQuiz.size}: ${resultMessage(s, kotlinQuiz.size)}")
    }
    println(score(listOf(1, 2), kotlinQuiz))     // unanswered questions count as wrong
}
```

Output

```
5/5: Perfect! You're a Kotlin star.
3/5: Good job! Review the ones you missed.
0/5: Keep practising. You'll get there!
2
```

3. Navigation and screens

QuizApp.kt

```
@Serializable object QuizHomeRoute
@Serializable object QuizRoute
@Serializable data class ResultRoute(val score: Int, val total: Int)

@Composable
fun QuizNavHost() {
    val navController = rememberNavController()
    NavHost(navController = navController, startDestination = QuizHomeRoute) {
        composable<QuizHomeRoute> {
            QuizHomeScreen(
                questionCount = kotlinQuiz.size,
                onStart = { navController.navigate(QuizRoute) }
            )
        }
        composable<QuizRoute> {
            QuizScreen(
                questions = kotlinQuiz,
                onFinished = { finalScore ->
```

```
                    navController.navigate(ResultRoute(finalScore, kotlinQuiz.size)) {
                        popUpTo<QuizHomeRoute>()          // remove the quiz: Back from results goes Home
                    }
                }
            )
        }
        composable<ResultRoute> { entry ->
            val route: ResultRoute = entry.toRoute()
            ResultScreen(
                score = route.score,
                total = route.total,
                onPlayAgain = {
                    navController.navigate(QuizRoute) { popUpTo<QuizHomeRoute>() }
                },
                onHome = { navController.popBackStack() }
            )
        }
    }
}

@Composable
fun QuizHomeScreen(questionCount: Int, onStart: () -> Unit) {
    Column(
        modifier = Modifier.fillMaxSize().padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Kotlin Quiz", style = MaterialTheme.typography.displaySmall)
        Spacer(Modifier.height(8.dp))
        Text("$questionCount questions about what you've learned")
        Spacer(Modifier.height(32.dp))
        Button(onClick = onStart, modifier = Modifier.fillMaxWidth()) { Text("Start quiz") }
    }
}

@Composable
fun QuizScreen(questions: List<Question>, onFinished: (Int) -> Unit) {
    var index by rememberSaveable { mutableStateOf(0) }
    var answers by rememberSaveable { mutableStateOf(listOf<Int>()) }
    var selected by rememberSaveable { mutableStateOf<Int?>(null) }
    val question = questions[index]
    val isLast = index == questions.lastIndex

    Column(modifier = Modifier.fillMaxSize().padding(24.dp)) {
        Text("Question ${index + 1} of ${questions.size}", style = MaterialTheme.typography.labelLarge)
        LinearProgressIndicator(
            progress = { (index + 1) / questions.size.toFloat() },
            modifier = Modifier.fillMaxWidth().padding(vertical = 8.dp)
        )
        Text(question.text, style = MaterialTheme.typography.headlineSmall)
        Spacer(Modifier.height(16.dp))

        question.options.forEachIndexed { optionIndex, option ->
            OptionRow(
                text = option,
                selected = selected == optionIndex,
                onClick = { selected = optionIndex }
            )
        }

        Spacer(Modifier.weight(1f))
        Button(
            onClick = {
                val newAnswers = answers + (selected ?: -1)
                if (isLast) {
                    onFinished(score(newAnswers, questions))
                } else {
                    answers = newAnswers
```

```
                    index++
                    selected = null
                }
            },
            enabled = selected != null,
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(if (isLast) "Finish" else "Next")
        }
    }
}

@Composable
fun OptionRow(text: String, selected: Boolean, onClick: () -> Unit) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 4.dp)
            .clip(MaterialTheme.shapes.medium)
            .background(
                if (selected) MaterialTheme.colorScheme.primaryContainer
                else MaterialTheme.colorScheme.surfaceVariant
            )
            .clickable(onClick = onClick)
            .padding(16.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        RadioButton(selected = selected, onClick = onClick)
        Text(text)
    }
}

@Composable
fun ResultScreen(score: Int, total: Int, onPlayAgain: () -> Unit, onHome: () -> Unit) {
    Column(
        modifier = Modifier.fillMaxSize().padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("$score / $total", style = MaterialTheme.typography.displayLarge)
        Spacer(Modifier.height(8.dp))
        Text(resultMessage(score, total), textAlign = TextAlign.Center)
        Spacer(Modifier.height(32.dp))
        Button(onClick = onPlayAgain, modifier = Modifier.fillMaxWidth()) { Text("Play again") }
        OutlinedButton(onClick = onHome, modifier = Modifier.fillMaxWidth()) { Text("Home") }
    }
}
```

4. Understand the key decisions

Decision	Why
ResultRoute(score, total) carries two Ints	Small, simple values: perfect route arguments
popUpTo<QuizHomeRoute>() when finishing	Removes the Quiz screen, so Back on Results goes Home, not to the last question
rememberSaveable for index, answers, selected	Rotating the phone mid-quiz keeps your progress
answers + selected (a new list)	Immutable list in state (Week 6, Day 2)
score() and resultMessage() are plain functions	Logic outside the UI, tested with real output above
Button enabled = selected != null	Users can't skip a question without choosing

Tip: Next week you'll move the quiz state (index, answers, score) into a **ViewModel**, which is how a team would build it.

5. Show it in MainActivity

MainActivity.kt (Android only)

```
setContent {
    Week7PracticeTheme {
        Surface(modifier = Modifier.fillMaxSize()) {
            QuizNavHost()
        }
    }
}
```

6. Project challenges

- Show whether each answer was right or wrong (green or red) for one second before moving to the next question.
- Add a **ReviewRoute** from the results screen that lists every question with the correct answer.
- Add a top app bar to QuizScreen with a close (X) icon that asks "Quit the quiz?" in a dialog.
- Shuffle the questions each time with `remember { kotlinQuiz.shuffled() }`.

Week 7 checklist

I can...	Done?
Set up NavHost with type-safe routes	[ ]
Navigate forward and back	[ ]
Pass arguments with route data classes and read them with toRoute()	[ ]
Explain why to pass IDs instead of objects	[ ]
Use popUpTo, inclusive and launchSingleTop correctly	[ ]
Build a bottom navigation bar with saved tab state	[ ]
Organise navigation with feature extension functions and nested graphs	[ ]
Keep screens free of NavController (lambdas only)	[ ]
Build the multi-screen Quiz app	[ ]

Quick Quiz

Q1. Why does finishing the quiz use popUpTo()?

Q2. Why is the score passed as a route argument instead of the whole answers list?

Q3. What keeps the quiz progress when the phone rotates?

Q4. Which functions in the project have no UI and can be tested with plain Kotlin?

Q5. Why can't the user press Next without choosing an option?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: Navigation Basics

Exercise 1

```
@Serializable object HomeRoute
@Serializable object ContactRoute

@Composable
fun AppNavHost() {
    val navController = rememberNavController()
    NavHost(navController, startDestination = HomeRoute) {
        composable<HomeRoute> {
            Button(onClick = { navController.navigate(ContactRoute) }) { Text("Contact us") }
        }
        composable<ContactRoute> {
            Text("Email: hello@example.com", modifier = Modifier.padding(16.dp))
        }
    }
}
```

Exercise 2

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun AboutScreen(onBack: () -> Unit) {
    Scaffold(topBar = {
        TopAppBar(
            title = { Text("About") },
            navigationIcon = {
                IconButton(onClick = onBack) {
                    Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Back")
                }
            }
        )
    }) { innerPadding ->
        Text("Made by Anjana", modifier = Modifier.padding(innerPadding).padding(16.dp))
    }
}
```

Exercise 3

```
@Composable
fun HomeScreen(onAboutClick: () -> Unit) {
    Button(onClick = onAboutClick) { Text("About this app") }
}

@Preview(showBackground = true)
@Composable
fun HomeScreenPreview() {
    HomeScreen(onAboutClick = { })      // no NavController needed
}
```

Quiz answers

Q1. It shows the current screen and defines all the routes (the navigation graph).

Q2. Pushes the About screen on top of the back stack.

Q3. It doesn't carry any data, so a single object is enough.

Q4. They stay previewable, reusable and testable, and navigation stays in one place.

Q5. The Kotlin serialization plugin (for @Serializable).

Day 2: Passing Data Between Screens

Exercise 1

```
data class Student(val roll: Int, val name: String)
val students = (1..5).map { Student(it, "Student $it") }

@Serializable object StudentListRoute
@Serializable data class StudentDetailRoute(val rollNumber: Int)

@Composable
fun StudentNavHost() {
    val navController = rememberNavController()
    NavHost(navController, startDestination = StudentListRoute) {
        composable<StudentListRoute> {
            LazyColumn {
                items(students, key = { it.roll }) { s ->
                    Text(s.name, Modifier.fillMaxWidth()
                        .clickable { navController.navigate(StudentDetailRoute(s.roll)) }.padding(16.dp))
                }
            }
        }
        composable<StudentDetailRoute> { entry ->
            val route: StudentDetailRoute = entry.toRoute()
            Text("Roll number ${route.rollNumber}: ${students.find { it.roll == route.rollNumber }?.name}")
        }
    }
}
```

Exercise 2

```
@Serializable data class ArticleRoute(val articleId: Int, val fromNotification: Boolean = false)

@Composable
fun ArticleScreen(articleId: Int, fromNotification: Boolean) {
    Column(modifier = Modifier.padding(16.dp)) {
        if (fromNotification) {
            Surface(color = MaterialTheme.colorScheme.secondaryContainer) {
                Text("Opened from a notification", modifier = Modifier.padding(8.dp))
            }
        }
        Text("Article $articleId", style = MaterialTheme.typography.headlineSmall)
    }
}
```

Exercise 3

```
// navController.navigate(ProductDetailRoute(99)) -> allProducts.find { it.id == 99 } is null,
// so ProductDetailScreen(product = null) shows "Product not found".
@Composable
fun NotFoundDemo() { Text("Product not found") }
```

Quiz answers

Q1. Make it a @Serializable data class whose properties are the arguments.

Q2. val route: MyRoute = backStackEntry.toRoute()

Q3. IDs are small and the next screen loads fresh data; objects can be large, stale or lost.

Q4. Give it a default value (and make it nullable if null is allowed).

Q5. It doesn't compile. Type-safe routes are checked by the compiler.

Day 3: The Back Stack

Exercise 1

```
@Serializable object WelcomeRoute
@Serializable object LanguageRoute
@Serializable object HomeRoute

@Composable
fun OnboardingNavHost() {
    val navController = rememberNavController()
    NavHost(navController, startDestination = WelcomeRoute) {
        composable<WelcomeRoute> {
            Button(onClick = { navController.navigate(LanguageRoute) }) { Text("Get started") }
        }
        composable<LanguageRoute> {
            Button(onClick = {
                navController.navigate(HomeRoute) { popUpTo<WelcomeRoute> { inclusive = true } }
            }) { Text("Continue") }
        }
        composable<HomeRoute> { Text("Home") }
    }
}
```

Exercise 2

```
@Serializable object LoginRoute

fun logout(navController: NavHostController) {
    navController.navigate(LoginRoute) {
        popUpTo(navController.graph.findStartDestination().id) { inclusive = true }
        launchSingleTop = true
    }
}

@Composable
fun SettingsScreen(navController: NavHostController) {
    // (inside a NavHost you would pass a lambda instead of navController)
    Button(onClick = { logout(navController) }) { Text("Log out") }
}
```

Exercise 3

```
// With launchSingleTop = true, the second tap doesn't add another Settings screen.
// One Back press returns to the previous screen.
@Serializable object SettingsRoute
fun openSettings(navController: NavHostController) {
    navController.navigate(SettingsRoute) { launchSingleTop = true }
}
```

Quiz answers

Q1. The app closes (the Activity finishes).

Q2. Removes LoginRoute and everything above it before navigating.

Q3. So the same screen isn't added twice when the user taps quickly.

Q4. navigateUp() (or popBackStack()).

Q5. Intercepting the system Back button, for example to confirm discarding changes.

Day 4: Bottom Navigation

Exercise 1

```
@Serializable object NotificationsRoute
// Add to topLevelDestinations:
// TopLevelDestination(NotificationsRoute, "Alerts", Icons.Default.Notifications)
// and to the NavHost:
// composable<NotificationsRoute> { TabContent("Notifications") }
@Composable
fun NotificationsTab() { Text("Notifications") }
```

Exercise 2

```
@Composable
fun FeedList() {
    LazyColumn(Modifier.fillMaxSize()) {
        items((1..30).toList(), key = { it }) { Text("Post $it", Modifier.padding(16.dp)) }
    }
}
// With saveState/restoreState the scroll position comes back when you return to the tab.
```

Exercise 3

```
@Serializable data class PostDetailRoute(val postId: Int)

@Composable
fun showBottomBar(navController: NavHostController): Boolean {
    val entry by navController.currentBackStackEntryAsState()
    return entry?.destination?.hasRoute(PostDetailRoute::class) != true
}
```

Quiz answers

Q1. It reads the current back stack entry and checks whether its destination matches the tab's route.

Q2. Saves the state of the tab you're leaving so it can be restored later.

Q3. So switching tabs doesn't build a huge back stack.

Q4. So the screens aren't hidden behind the bottom bar.

Q5. 3 to 5.

Day 5: Structuring Navigation in an App

Exercise 1

```
data class Product(val id: Int, val name: String)
@Serializable object ProductListRoute
@Serializable data class ProductDetailRoute(val productId: Int)

fun NavGraphBuilder.shopScreens(onProductClick: (Int) -> Unit, onBack: () -> Unit) {
    composable<ProductListRoute> {
        Button(onClick = { onProductClick(1) }) { Text("Open product 1") }
    }
    composable<ProductDetailRoute> { entry ->
        val route: ProductDetailRoute = entry.toRoute()
        Column { Text("Product ${route.productId}"); TextButton(onClick = onBack) { Text("Back") } }
    }
}
```

Exercise 2

```
@Serializable object CheckoutGraph
@Serializable object AddressRoute
@Serializable object PaymentRoute
@Serializable object ConfirmRoute
@Serializable object OrderPlacedRoute

fun NavGraphBuilder.checkout(navController: NavHostController) {
    navigation<CheckoutGraph>(startDestination = AddressRoute) {
        composable<AddressRoute> { Button(onClick = { navController.navigate(PaymentRoute) }) { Text("Next") } }
        composable<PaymentRoute> { Button(onClick = { navController.navigate(ConfirmRoute) }) { Text("Next") } }
        composable<ConfirmRoute> {
            Button(onClick = {
                navController.navigate(OrderPlacedRoute) { popUpTo<CheckoutGraph> { inclusive = true } }
            }) { Text("Place order") }
        }
    }
    composable<OrderPlacedRoute> { Text("Order placed!") }
}
```

Quiz answers

Q1. To keep AppNavHost short and let each feature own its destinations.

Q2. A group of related destinations with its own start destination and route.

Q3. popUpTo { inclusive = true }

Q4. In that screen's own Scaffold.

Q5. A link (URL or notification) that opens a specific screen in the app.

Day 6: Weekly Project: Multi-Screen Quiz App

Quiz answers

Q1. So the Quiz screen is removed and Back from Results returns to Home.

Q2. Route arguments should be small, simple values.

Q3. rememberSaveable for index, answers and selected.

Q4. score() and resultMessage().

Q5. The button is enabled only when selected != null.

Next: When you finish this week, tell your mentor and ask for the **Week 8** material. Great work, Anjana!